Projet : Inférence médicale embarquée

Quantification et déploiement de modèles de deep learning
sur des machines virtuelles à ressources contraintes

Module : Systèmes embarqués et objets connectés – Master Data Science
ENS Martil

Prof: Said Ohamouddou

1er mars 2026

Table des matières

1 Contexte et objectifs **2**

2 Notions fondamentales **2**
2.1 La quantification (*Quantization*) . . . 2
2.2 L'élagage (*Pruning*) . . . 3
2.3 Différence entre quantification et élagage . . . 3

3 Environnement de travail **4**
3.1 Machine hôte . . . 4
3.2 Machines virtuelles (VM) . . . 4

4 Jeu de données médical **4**

5 Phases de réalisation **5**
5.1 Phase 1 — Modèle de base (*Baseline*) . . . 5
5.2 Phase 2 — Application des techniques d'optimisation . . . 5
5.3 Phase 3 — Déploiement sur les trois VM . . . 6
5.4 Phase 4 — Sélection de la meilleure technique par VM . . . 6
5.5 Phase 5 — Intelligence collective . . . 7
5.6 Phase 6 — Supervision avec ThingsBoard . . . 7

6 Livrables attendus **8**
6.1 Rapport technique (10–12 pages) . . . 8
6.2 Dépôt GitHub structuré . . . 8
6.3 Vidéo de démonstration du projet . . . 9
6.4 Tableau de suivi partagé (Google Sheets) . . . 9

7 Grille d'évaluation **9**

8 Calendrier **10**

1 Contexte et objectifs

Objectif pédagogique

Ce projet vise à vous faire comprendre les compromis entre **performance des modèles**, **efficacité computationnelle** et **contraintes matérielles** dans un contexte d'intelligence artificielle embarquée. Vous apprendrez à optimiser des modèles de deep learning pour les déployer sur des environnements à ressources limitées, en évaluant l'impact de chaque technique d'optimisation [1].

Les dispositifs embarqués (objets connectés, systèmes médicaux portables, etc.) disposent de ressources matérielles fortement limitées : mémoire réduite, puissance de calcul modeste, consommation énergétique contrainte. Or, les modèles de deep learning modernes sont souvent trop lourds pour être déployés tels quels sur ces appareils.

Ce projet vous amène à :

1. Entraîner un modèle de base sur un jeu de données médical public.
2. Appliquer des techniques d'optimisation (quantification et élagage) pour réduire la taille et le coût d'inférence du modèle [1].
3. Déployer et évaluer ces modèles optimisés dans trois environnements virtuels simulant des contraintes matérielles croissantes.
4. Combiner les prédictions des trois machines virtuelles en un système d'intelligence collective.
5. Superviser l'ensemble via une plateforme IoT (ThingsBoard).

Tout le travail est réalisé en simulation sur votre machine hôte (8 Go de RAM), en utilisant des machines virtuelles (VM) configurées avec des limites strictes de ressources.

2 Notions fondamentales

2.1 La quantification (*Quantization*)

Définition — Quantification

La **quantification** consiste à réduire la précision numérique utilisée pour représenter les poids et/ou les activations d'un réseau de neurones. Par exemple, au lieu de stocker chaque valeur sur 32 bits (virgule flottante, `float32`), on la représente sur 8 bits (`int8`) ou même 4 bits (`int4`).

Effets attendus : réduction de la taille du modèle (jusqu'à 4×), accélération de l'inférence, diminution de la consommation mémoire. La contrepartie est une légère perte de précision (*accuracy*).

On distingue plusieurs variantes, présentées en ordre croissant de complexité :

Q1. **Quantification dynamique** : les poids sont quantifiés une fois après l'entraînement ; les activations sont quantifiées à la volée pendant l'inférence. Simple à mettre en œuvre, sans calibration.

Q2. **Quantification statique post-entraînement (PTQ)** : les poids *et* les activations sont quantifiés après l'entraînement, en utilisant un petit ensemble de calibration pour estimer les plages de valeurs des activations.

Q3. Quantification consciente de l'entraînement (QAT) : la quantification est simulée *pendant* l'entraînement afin que le modèle s'y adapte. Donne de meilleurs résultats mais nécessite de ré-entraîner le modèle.

Q4. Quantification poids uniquement (*Weight-Only*) : seuls les poids sont quantifiés (en `int4` ou `int8`) ; les activations restent en virgule flottante.

Q5. Quantification à précision mixte : différentes couches utilisent des largeurs de bits différentes selon leur sensibilité à la perte de précision.

2.2 L'élagage (*Pruning*)

Définition — Élagage

L'**élagage** consiste à supprimer des paramètres jugés peu importants dans un réseau de neurones, afin d'en réduire la complexité. Un poids (ou un groupe de poids) est mis à zéro s'il contribue peu à la prédiction finale.

Effets attendus : modèle plus léger (moins de paramètres), potentiellement plus rapide selon la structure de l'élagage. La contrepartie est aussi une légère perte de précision, qui peut être atténuée par un ré-entraînement (*fine-tuning*) après élagage.

On distingue :

P1. Élagage non structuré : des poids *individuels* sont mis à zéro, indépendamment de leur position. Produit des matrices creuses (*sparse*), difficiles à accélérer sans matériel spécialisé.

P2. Élagage structuré : des groupes entiers (filtres, canaux, couches) sont supprimés. Le réseau résultant est plus petit et directement plus rapide sur du matériel standard.

P3. Élagage basé sur la magnitude : les poids dont la valeur absolue est la plus faible sont supprimés en premier, de manière itérative, selon l'hypothèse qu'ils ont le moins d'impact sur les prédictions.

2.3 Différence entre quantification et élagage

Remarque

Ces deux techniques visent toutes deux à **alléger un modèle**, mais elles agissent sur des aspects différents [1] :

Aspect	Quantification	Élagage
Levier	Précision numérique des valeurs	Nombre de paramètres
Mécanisme	Réduire les bits (`float32` → `int8`)	Supprimer des poids ou des neurones
Structure	Inchangée (même architecture)	Modifiée (modèle plus petit)
Impact mémoire	Fort (taille ÷ 2–8×)	Variable (selon taux d'élagage)
Impact vitesse	Élevé sur CPU/NPU	Élevé surtout pour l'élagage structuré

En pratique, les deux techniques sont **complémentaires** et peuvent être combinées.

3 Environnement de travail

3.1 Machine hôte

Votre machine personnelle (8 Go de RAM) sert à l'entraînement des modèles et à l'exécution des machines virtuelles.

3.2 Machines virtuelles (VM)

Les trois VM simulent des dispositifs embarqués à ressources croissantes. Elles sont créées avec **VirtualBox** (ou des conteneurs Docker avec `-cpus` et `-memory`) sous **Raspberry Pi OS 64 bits** ou une image Debian légère.

VM	CPU (cœurs)	RAM	Profil représenté
VM1	1 cœur	500 Mo	Dispositif très contraint (capteur IoT, bas de gamme)
VM2	2 cœurs	1 Go	Dispositif intermédiaire (passerelle IoT)
VM3	2 cœurs	2 Go	Dispositif plus capable (edge server léger)

Remarque

Vérifiez que la somme de la RAM allouée (500 Mo + 1 Go + 2 Go = 3,5 Go) reste dans les limites de votre machine hôte lorsque les VM tournent simultanément. Prévoyez 2–3 Go pour le système hôte et les outils de développement.

Livrables d'infrastructure à fournir :

- Scripts de création et de configuration des VM (Vagrantfile ou `docker-compose.yml`).
- Script de vérification des ressources (`check_resources.sh`).

4 Jeu de données médical

Choisissez **un seul** jeu de données public parmi les catégories suivantes :

Catégorie	Exemples
Radiographies thoraciques	NIH Chest X-ray, CheXpert, COVID-19 Radiography
Signaux ECG	PTB-XL, MIT-BIH Arrhythmia
IRM cérébrales	Brain MRI Segmentation, IXI
Histopathologie	PatchCamelyon, LC25000

Critères obligatoires :

- Au moins 10 000 échantillons.
- Division train / validation / test : 70 % / 15 % / 15 %.
- Documentation claire des classes (au moins 2 classes).

Livrables à fournir : lien de téléchargement, script de prétraitement (*preprocessing*), description des classes et de la distribution des données.

5 Phases de réalisation

5.1 Phase 1 — Modèle de base (*Baseline*)

Objectif pédagogique

Comprendre le point de départ avant toute optimisation, et se familiariser avec les métriques de performance.

1. Choisir **une architecture** adaptée à votre tâche (ex. MobileNetV2, ResNet18, EfficientNet-B0).
2. Entraîner le modèle sur le jeu d'entraînement, valider sur le jeu de validation.
3. Évaluer sur le jeu de test et mesurer :
   - **Précision** (accuracy, F1-score).
   - **Taille du modèle** (Mo).
   - **Temps d'inférence moyen** (ms) sur 100 images.
   - **Consommation mémoire RAM** pendant l'inférence.
4. Comparer vos résultats avec au moins **2 articles de référence** portant sur la même tâche.

Livrables : script d'entraînement, fichier de poids (`.pt` ou `.h5`), tableau de résultats, bibliographie commentée.

5.2 Phase 2 — Application des techniques d'optimisation

Objectif pédagogique

Appliquer chacune des 8 techniques d'optimisation et mesurer empiriquement leurs effets sur la taille, la vitesse et la précision du modèle.

Pour chacune des 8 techniques (Q1 à Q5 et P1 à P3), appliquer l'optimisation au modèle de base et remplir le tableau suivant :

ID	Technique	Précision	Taille (Mo)	Compression	Inférence (ms)
Q1	Quantification dynamique				
Q2	Quantification statique (PTQ)				
Q3	QAT				
Q4	Weight-Only				
Q5	Précision mixte				
P1	Élagage non structuré				
P2	Élagage structuré				
P3	Élagage par magnitude				

Livrables : un script Python par technique, tableau comparatif complet, graphiques (taille vs précision, vitesse vs précision).

5.3 Phase 3 — Déploiement sur les trois VM

Objectif pédagogique

Comprendre l'impact des contraintes matérielles sur les performances réelles, et identifier quelles techniques sont viables selon la VM cible.

Procédure

Pour chaque combinaison (VM × technique), procéder comme suit :

1. Transférer le modèle optimisé sur la VM (via `scp` ou un répertoire partagé).
2. Vérifier que le modèle tient en RAM. Si ce n'est pas le cas, noter `OOM` (*Out of Memory*) dans la matrice.
3. Si le déploiement est possible, exécuter **10 inférences** et enregistrer :
   - Temps d'inférence moyen et écart-type (ms).
   - Utilisation CPU (%).
   - RAM consommée (Mo).
   - Précision sur le jeu de test.

Matrice de résultats (3×8)

	Q1	Q2	Q3	Q4	Q5	P1	P2	P3
VM1								
VM2								
VM3								

Livrables : scripts de déploiement et de mesure, matrice complète au format CSV.

5.4 Phase 4 — Sélection de la meilleure technique par VM

Objectif pédagogique

Apprendre à définir des critères de sélection adaptés aux contraintes spécifiques de chaque dispositif.

Pour chaque VM, calculer un **score pondéré** qui reflète ses priorités :

VM	Pondération du score
VM1 (500 Mo)	RAM (0,40) + CPU (0,40) + précision (0,20)
VM2 (1 Go)	RAM (0,30) + vitesse (0,30) + précision (0,40)
VM3 (2 Go)	précision (0,60) + vitesse (0,25) + RAM (0,15)

Justifier le choix de la meilleure technique pour chaque VM en expliquant pourquoi elle correspond au profil de la VM.

Livrables : tableau de sélection, paragraphe de justification par VM.

5.5 Phase 5 — Intelligence collective

Objectif pédagogique

Découvrir comment combiner les prédictions de plusieurs nœuds pour améliorer la fiabilité d'un système distribué.

Architecture

Chaque VM déploie la **meilleure technique** identifiée en Phase 4. Un **orchestrateur** (script Python tournant sur la machine hôte) distribue les requêtes et agrège les prédictions.

$$\textbf{Patient} \longrightarrow \textbf{Orchestrateur} \longrightarrow \begin{array}{c}\text{VM1}\\ \text{VM2}\\ \text{VM3}\end{array} \longrightarrow \textbf{Agrégateur} \longrightarrow \textbf{Diagnostic final}$$

Mécanismes à implémenter

1. **Vote pondéré :** chaque VM vote avec un poids proportionnel à sa précision historique et à la confiance (*softmax*) de sa prédiction.
2. **Validation par confiance :** si la confiance du vote collectif est inférieure à 70 %, toutes les VM sont sollicitées à nouveau avec un sous-ensemble augmenté de l'image.
3. **Équilibrage de charge :** si une VM dépasse 85 % d'utilisation CPU ou 90 % de RAM, l'orchestrateur redirige la requête vers une VM moins chargée.

Évaluation

Mesurer sur au moins 10 exemples de test :

— Précision collective vs précision du meilleur modèle individuel.

— Taux de consensus (proportion de cas où les 3 VM sont d'accord).

— Gain moyen apporté par la validation par confiance.

Livrables : script orchestrateur, tableaux de comparaison collectif vs individuel.

5.6 Phase 6 — Supervision avec ThingsBoard

Objectif pédagogique

Mettre en pratique l'intégration IoT : envoi de télémétrie via MQTT et création de tableaux de bord de supervision.

Déployer un serveur **ThingsBoard Community Edition** en local (via Docker) et y connecter les VM via **MQTT**. La documentation officielle est disponible à l'adresse :

https://thingsboard.io/docs/

Format de télémétrie

Chaque VM publie un message JSON après chaque inférence :

```
{
  "vm_id"             : "VM1",
  "timestamp"         : "2024-01-15T14:32:01Z",
```

```
  "technique"         : "Quantification dynamique",
  "prediction"        : "Pneumonie",
  "confidence"        : 0.87,
  "inference_time_ms" : 142,
  "cpu_usage_pct"     : 72,
  "ram_usage_mb"      : 310,
  "patient_id"        : "P-2024-042"
}
```

Tableaux de bord à créer

1. **Monitoring temps réel :** prédictions en direct, diagnostic collectif, taux de confiance.
2. **Ressources système :** graphiques CPU et RAM par VM en temps réel.
3. **Comparaison des techniques :** *heatmap* 3×3 des scores.
4. **Intelligence collective :** précision collective vs individuelle, taux de consensus.
5. **Alertes :** notification si latence > 300 ms, RAM $> 90\,\%$, ou précision collective $< 80\,\%$.

Livrables : client MQTT Python, fichiers JSON d'import des dashboards, captures d'écran annotées.

6 Livrables attendus

6.1 Rapport technique (10–12 pages)

Structure attendue :

1. Introduction et contexte applicatif.
2. Présentation du jeu de données et du prétraitement.
3. Modèle de base : architecture, résultats, comparaison littérature.
4. Techniques d'optimisation : description, résultats, analyse comparative.
5. Des capteurs d'écran
6. Déploiement sur les VM : matrice des résultats, analyse.
7. Intelligence collective : architecture, résultats, gains mesurés.
8. Supervision ThingsBoard : captures et description des dashboards.
9. Conclusion et perspectives.

6.2 Dépôt GitHub structuré

```
projet-embarque/
 README.md               # Description du projet et instructions
 environment/            # Vagrantfile ou docker-compose.yml
    check_resources.sh
 dataset/                # Script de téléchargement et prétraitement
 baseline/               # Entraînement et évaluation du modèle de base
 optimization/           # Un sous-dossier par technique (Q1-Q5, P1-P3)
    Q1_dynamic_quant/
    Q2_static_ptq/
    ...
 deployment/             # Scripts de déploiement sur chaque VM
 collective/             # Orchestrateur et mécanismes de vote
```

```
thingsboard/            # Client MQTT et JSON des dashboards
results/                # Tableaux CSV et figures
report/                 # Rapport final en PDF
```

6.3 Vidéo de démonstration du projet

— Seulement pour Phase 6 — Supervision avec ThingsBoard

6.4 Tableau de suivi partagé (Google Sheets)

Chaque groupe doit renseigner ses résultats dans le tableau de suivi commun accessible via le lien ci-dessous. **Ce tableau est obligatoire** et doit être complété au fur et à mesure de l'avancement du projet.

https://docs.google.com/spreadsheets/d/1f-uBdE_nkICrYzGhdRvOpAj7JJKjhTTr4KCLelvXNNM/edit?usp=sharing

Colonnes à renseigner :

Groupe	Membres	Dataset	Architecture	Tech. VM1

Tech. VM2	Tech. VM3	Gain collectif (%)	URL GitHub

Remarque

Assurez-vous que votre dépôt GitHub est **public** et que l'URL ThingsBoard est accessible avant la soumission finale.

7 Grille d'évaluation

Critère	Description	Points
Modèle de base	Entraînement correct, comparaison avec la littérature	10
Techniques d'optimisation	Implémentation et analyse des 8 techniques	25
Déploiement sur les VM	Matrice 3×8 complète, gestion des cas OOM	15
Sélection par VM	Justification claire et cohérente des choix	15
Intelligence collective	Fonctionnalité, vote pondéré, analyse des gains	20
ThingsBoard	Dashboards fonctionnels, alertes, télémétrie	10
Rapport	Clarté, rigueur, qualité rédactionnelle	5
Total		**100**
Bonus	Combinaison quant. + élagage, XAI (*Grad-CAM*), application de visualisation	+10

8 Calendrier

Le tableau ci-dessous présente la progression attendue sur un nombre de séances de travail. **Les dates précises seront communiquées par l'enseignant.**

Séance(s)	Travaux à réaliser	Livrable intermédiaire
1–2	Mise en place des VM, choix du dataset, entraînement du baseline	Rapport de baseline
3–4	Application des 5 techniques de quantification (Q1–Q5)	Scripts + tableau
5	Application des 3 techniques d'élagage (P1–P3)	Scripts + tableau
6	Déploiement sur les 3 VM, remplissage de la matrice	Matrice CSV
7	Sélection et justification de la meilleure technique par VM	Tableau de sélection
8–9	Implémentation de l'intelligence collective	Script orchestrateur
10	Intégration ThingsBoard, rédaction du rapport final	Rapport PDF + GitHub

Référence

[1] Liang, T., Glossner, J., Wang, L., Shi, S., & Zhang, X. (2021). *Pruning and quantization for deep neural network acceleration : A survey.* Neurocomputing, 461, 370–403. https://doi.org/10.1016/j.neucom.2021.07.045

Projet à réaliser en groupes de 3 étudiants. Toute soumission doit inclure le lien vers le dépôt GitHub public et le tableau de suivi Google Sheets complété.